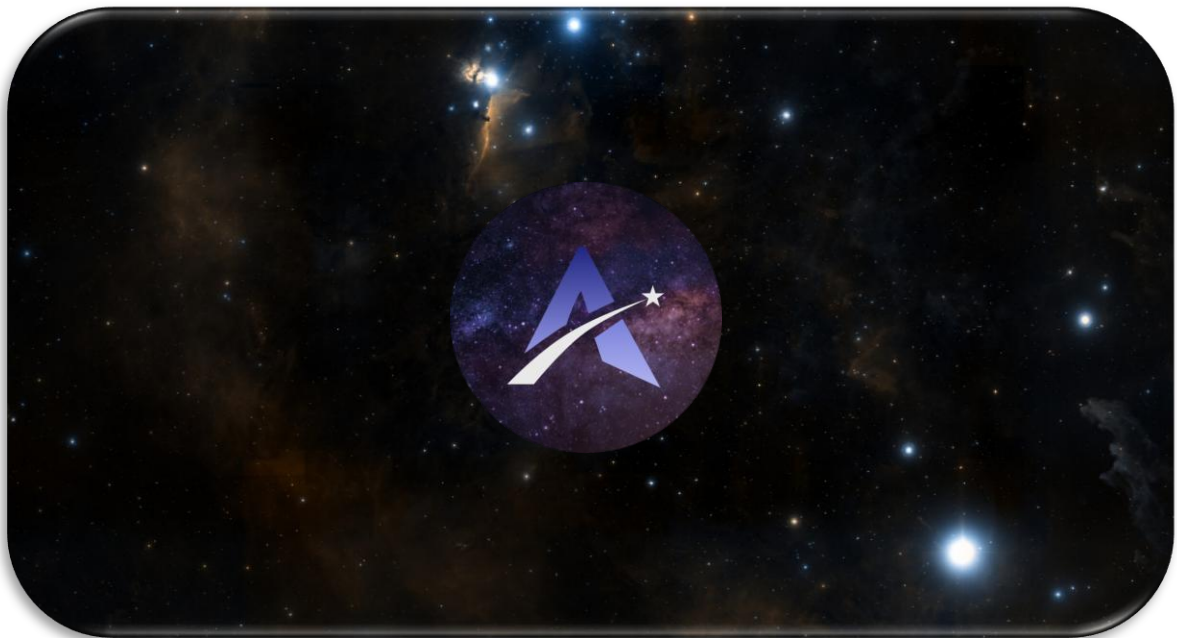




# Travail de fin d'études Andromesky

*Exploration interactive du ciel grâce à une application web développée avec Angular*



Florian Marchal  
Année académique 2025-2026  
Développeur Web Front-End



[fjolink1er.github.io/andromesky/](https://fjolink1er.github.io/andromesky/)

## Table des matières

|                                                              |    |
|--------------------------------------------------------------|----|
| Introduction .....                                           | 2  |
| Détails de l'application .....                               | 3  |
| Description du projet.....                                   | 3  |
| Fonctionnalités .....                                        | 4  |
| 1. Exploration du ciel .....                                 | 4  |
| 2. Recherche d'objets .....                                  | 4  |
| 3. Consultation des informations.....                        | 4  |
| 4. Enrichissement des informations.....                      | 4  |
| 5. Mode Quiz .....                                           | 5  |
| 6. Difficultés.....                                          | 5  |
| 7. Système de score .....                                    | 5  |
| 8. Interface utilisateur .....                               | 6  |
| Priorisation des fonctionnalités .....                       | 6  |
| Analyse de la priorisation .....                             | 8  |
| 4. Justification de la priorisation.....                     | 8  |
| Technologies utilisées et justifications.....                | 8  |
| 1. Angular .....                                             | 8  |
| 2. TypeScript.....                                           | 9  |
| 3. Aladin Lite.....                                          | 9  |
| 4. APIs externes .....                                       | 10 |
| 5. Données locales (JSON) .....                              | 11 |
| 6. Outils de développement .....                             | 11 |
| 7. Conclusion sur les choix technologiques .....             | 11 |
| Outils .....                                                 | 11 |
| Outils de développement .....                                | 11 |
| Outils d'hébergement.....                                    | 14 |
| Conclusion sur les outils utilisés .....                     | 16 |
| Anticipation des problèmes et difficultés potentielles ..... | 16 |
| 1. Intégration de la bibliothèque Aladin Lite.....           | 17 |
| 2. Gestion des coordonnées astronomiques .....               | 17 |
| 3. Gestion des interactions utilisateur .....                | 17 |
| 4. Communication avec les API externes .....                 | 18 |
| 5. Gestion de la logique du quiz.....                        | 18 |
| 6. Gestion de l'expérience utilisateur .....                 | 19 |
| 7. Manque de connaissances sur certaines technologies .....  | 19 |

|                                                    |    |
|----------------------------------------------------|----|
| 8. Gestion du temps et des priorités.....          | 19 |
| 9. Conclusion sur les difficultés anticipées ..... | 20 |
| Architecture de l'application .....                | 20 |
| Composants.....                                    | 21 |
| Services .....                                     | 21 |
| Modèles de données .....                           | 22 |
| Ressources de données .....                        | 22 |
| Helpers et constantes .....                        | 23 |
| Interface générale .....                           | 24 |
| Exploration du ciel.....                           | 24 |
| Recherche.....                                     | 25 |
| Configuration du quiz .....                        | 26 |
| Quiz « Deviner l'objet » .....                     | 26 |
| Quiz « Localiser l'objet ».....                    | 27 |
| Résultats.....                                     | 28 |
| Difficultés rencontrées .....                      | 29 |
| Perspectives d'évolution.....                      | 30 |
| Conclusion.....                                    | 30 |

## Introduction

Je suis Florian Marchal, j'ai 26 ans et je suis passionné d'informatique et de science. Après quelques tentatives d'études supérieures et un service citoyen réalisé, je me suis tourné vers une formation qui réunis à la fois formateurs expérimentés, pratiques concrètes et possibilité de stage afin de poser un pied au sein du monde professionnel.

Dans le cadre de ma formation en développement web front-end, il m'a été demandé de réaliser un Travail de Fin d'Études. Ce projet a pour objectif de mettre en pratique les compétences acquises au cours de la formation, notamment en développement web, en conception d'interfaces interactives et en utilisation d'API externes.

Depuis mes débuts dans le développement, je me suis particulièrement intéressé à la création d'applications interactives et visuelles. La logique de programmation et le résultat visuel de son application m'attirent particulièrement, car ils permettent de rendre un projet concret tout en m'étant stimulant intellectuellement.

Par ailleurs, l'astronomie est un domaine qui suscite un intérêt particulier de ma part. L'idée de pouvoir représenter le ciel et ses objets sous forme d'application interactive m'a semblé être un excellent moyen de combiner mes centres d'intérêt avec mes compétences techniques.

C'est dans ce contexte que j'ai choisi de développer une application web permettant d'explorer le ciel de manière interactive. L'objectif principal du projet est de proposer une carte du ciel accessible

depuis un navigateur, enrichie de fonctionnalités pédagogiques telles qu'un mode d'exploration guidée et un système de quiz interactif.

L'application s'appuie sur des technologies web modernes, notamment Angular pour la structure de l'application, ainsi que l'intégration d'une bibliothèque de visualisation astronomique permettant d'afficher une carte du ciel dynamique. Des API externes, comme celles proposées par la NASA, sont également utilisées afin d'enrichir le contenu de l'application avec des données et des images réelles.

Ce projet vise donc à proposer une expérience à la fois interactive et éducative, permettant à la fois pour l'utilisateur de découvrir le ciel et pour moi un moyen de mettre en pratique des concepts de développement avancés.

## Détails de l'application

### Description du projet

Le projet **AndromeSky** est une application web de type **SPA (Single Page Application)** permettant d'explorer le ciel de manière interactive directement depuis un navigateur web. Son objectif est de rendre la découverte de l'astronomie plus accessible grâce à une interface moderne combinant visualisation, exploration et apprentissage.

L'application repose sur l'intégration de la bibliothèque **Aladin Lite**, qui permet d'afficher une carte du ciel interactive. L'utilisateur peut librement se déplacer sur la carte, zoomer, sélectionner des objets célestes et consulter les informations qui leur sont associées. Les principaux types d'objets présents dans l'application sont les constellations, les étoiles, les galaxies, les nébuleuses, les amas d'étoiles ainsi que certains autres objets remarquables.

Afin d'enrichir cette exploration, chaque objet est accompagné d'un panneau d'informations présentant ses principales caractéristiques, telles que son nom, son type, son catalogue d'origine ou encore sa constellation lorsqu'elle est connue. L'application récupère également des informations complémentaires provenant de services externes. Des images réelles issues des **NASA Open APIs** permettent d'illustrer certains objets célestes, tandis que l'API de **Wikipédia** fournit automatiquement une description synthétique afin d'offrir davantage de contexte scientifique.

En complément de l'exploration, AndromeSky propose un **mode Quiz** permettant à l'utilisateur de tester ses connaissances de manière ludique. Deux types de quiz sont disponibles. Le premier consiste à identifier un objet céleste parmi plusieurs propositions (QCM), tandis que le second demande de localiser directement un objet sur la carte du ciel. Plusieurs niveaux de difficulté sont proposés afin d'adapter les questions au niveau de connaissance de l'utilisateur. Un système de score permet de suivre les performances réalisées au cours d'une partie.

Les données astronomiques utilisées par l'application sont principalement stockées localement sous forme de fichiers JSON, garantissant un contrôle précis des objets proposés dans les différents modes de jeu. Ces données sont complétées par des services externes permettant d'enrichir l'expérience utilisateur sans alourdir la structure de l'application.

Ainsi, AndromeSky ne se limite pas à une simple carte du ciel interactive. L'application combine visualisation, informations pédagogiques et activités ludiques afin de proposer une expérience d'apprentissage progressive, destinée aussi bien aux personnes souhaitant découvrir l'astronomie qu'aux utilisateurs désirant approfondir leurs connaissances

## Fonctionnalités

L'application *AndromeSky* propose un ensemble de fonctionnalités visant à offrir une expérience interactive et pédagogique autour de l'exploration du ciel. Ces fonctionnalités sont organisées autour de plusieurs axes principaux : la visualisation, l'exploration guidée, le mode quiz et l'enrichissement des contenus.

---

### 1. Exploration du ciel

L'utilisateur peut explorer librement une carte du ciel interactive grâce à la bibliothèque **Aladin Lite**. Il est possible de naviguer dans le ciel, de zoomer sur les différentes régions célestes et de sélectionner les objets présents dans le catalogue de l'application.

Pour faciliter la découverte des différents objets, l'application propose également une navigation guidée à l'aide de boutons *Précédent* et *Suivant*, permettant de parcourir successivement l'ensemble du catalogue astronomique.

---

### 2. Recherche d'objets

Une barre de recherche permet de retrouver rapidement un objet astronomique à partir de son nom.

La recherche prend en compte les différents noms enregistrés pour chaque objet afin de faciliter leur localisation. Une fois l'objet sélectionné, la carte est automatiquement centrée sur celui-ci et le panneau d'informations est mis à jour.

---

### 3. Consultation des informations

Chaque objet astronomique possède une fiche descriptive affichée dans un panneau latéral.

Cette fiche présente notamment :

- son nom ;
- son type ;
- son catalogue d'origine ;
- sa constellation (si applicable) ;
- sa magnitude lorsque cette information est disponible.

Lorsqu'un objet appartient à une constellation, celle-ci est également mise en évidence directement sur la carte du ciel afin de faciliter son identification.

---

### 4. Enrichissement des informations

Afin de rendre l'exploration plus immersive, les informations locales sont complétées par deux services externes.

Les **NASA Open APIs** permettent d'afficher une image lorsqu'elle est disponible pour l'objet sélectionné.

L'API de **Wikipédia** fournit automatiquement une description synthétique de l'objet. Un système de recherche par alias et de filtrage des résultats est utilisé afin d'obtenir la description la plus pertinente possible malgré les ambiguïtés présentes sur Wikipédia.

---

## 5. Mode Quiz

L'application propose un mode Quiz destiné à vérifier les connaissances acquises durant l'exploration.

Deux modes de jeu sont disponibles.

### 5.1 Quiz « Identifier l'objet »

L'utilisateur doit reconnaître un objet astronomique affiché sur la carte parmi plusieurs propositions.

Chaque bonne réponse rapporte des points.

### 5.2 Quiz « Localiser l'objet »

L'utilisateur doit retrouver directement sur la carte la position de l'objet demandé.

Après avoir sélectionné un emplacement, il valide sa réponse afin que celle-ci soit comparée à la bonne position.

---

## 6. Difficultés

Les quiz proposent plusieurs niveaux de difficulté afin de s'adapter au niveau de connaissance de l'utilisateur.

Le filtrage des questions est réalisé en fonction des catégories d'objets astronomiques.

Par exemple :

- **Facile** : constellations et objets les plus connus ;
- **Moyen** : ajout des objets du catalogue Messier ;
- **Difficile** : ensemble du catalogue disponible.

Cette progression permet de découvrir progressivement des objets de plus en plus spécialisés.

---

## 7. Système de score

Chaque quiz utilise un système de score permettant de mesurer les performances du joueur.

À la fin d'une partie, l'application affiche notamment :

- le score obtenu ;

- le nombre de bonnes réponses ;
- le taux de réussite ;
- un message récapitulatif adapté au résultat obtenu.

Ces informations permettent au joueur de suivre sa progression au fil des différentes parties.

## 8. Interface utilisateur

L'interface de l'application est conçue pour être simple et intuitive. Elle se compose principalement de :

- Une zone centrale contenant la carte du ciel
- Un panneau d'information affichant les détails des objets et permettant d'accueillir les différents quiz

L'objectif est de permettre une utilisation fluide, sans surcharge d'informations, afin de mettre en valeur l'expérience utilisateur.

## Priorisation des fonctionnalités

Afin d'organiser le développement d'AndromeSky, les différentes fonctionnalités ont été priorisées selon la méthode **MoSCoW**. Cette méthode permet de classer les fonctionnalités en quatre catégories :

- **Must Have** : fonctionnalités indispensables au bon fonctionnement de l'application.
- **Should Have** : fonctionnalités importantes apportant une réelle valeur ajoutée, mais dont l'absence n'empêche pas l'utilisation de l'application.
- **Could Have** : fonctionnalités intéressantes pouvant être ajoutées si le temps le permet.
- **Won't Have** : fonctionnalités volontairement exclues du périmètre du projet pour cette version.

Cette approche a permis de concentrer les efforts de développement sur les fonctionnalités essentielles avant d'enrichir progressivement l'application.

| Fonctionnalité                          | Priorité         | Justification                                                      |
|-----------------------------------------|------------------|--------------------------------------------------------------------|
| Carte du ciel interactive (Aladin Lite) | <b>Must Have</b> | Élément central de l'application permettant l'exploration du ciel. |
| Navigation entre les objets             | <b>Must Have</b> | Permet de parcourir facilement le catalogue astronomique.          |
| Recherche d'objets                      | <b>Must Have</b> | Indispensable pour accéder rapidement à un objet précis.           |

| Fonctionnalité                                           | Priorité           | Justification                                                                     |
|----------------------------------------------------------|--------------------|-----------------------------------------------------------------------------------|
| Panneau d'informations                                   | <b>Must Have</b>   | Présente les informations principales de chaque objet astronomique.               |
| Chargement des catalogues JSON                           | <b>Must Have</b>   | Nécessaire au fonctionnement de l'ensemble de l'application.                      |
| Quiz « Identifier l'objet » (QCM)                        | <b>Must Have</b>   | Premier mode d'apprentissage interactif.                                          |
| Quiz « Localiser l'objet »                               | <b>Must Have</b>   | Complète le mode QCM en évaluant la connaissance de la position des objets.       |
| Difficultés du quiz                                      | <b>Must Have</b>   | Permet d'adapter les questions au niveau de l'utilisateur.                        |
| Système de score                                         | <b>Must Have</b>   | Fournit un retour immédiat sur les performances réalisées pendant un quiz.        |
| Images NASA                                              | <b>Should Have</b> | Enrichissent l'expérience utilisateur en illustrant les objets astronomiques.     |
| Descriptions Wikipédia                                   | <b>Should Have</b> | Complètent les informations locales avec une présentation synthétique des objets. |
| Mise en évidence des constellations                      | <b>Should Have</b> | Facilite le repérage visuel des objets dans leur environnement céleste.           |
| Recherche par alias                                      | <b>Should Have</b> | Améliore la pertinence des recherches et la récupération des données Wikipédia.   |
| Persistence des scores                                   | <b>Could Have</b>  | Fonctionnalité envisagée mais non implémentée dans le MVP.                        |
| Statistiques détaillées                                  | <b>Could Have</b>  | Amélioration possible du suivi des performances des utilisateurs.                 |
| Enrichissement du catalogue (NGC, Caldwell, planètes...) | <b>Could Have</b>  | Extension naturelle des données disponibles dans l'application.                   |
| SpaceGuessR                                              | <b>Could Have</b>  | Mode de jeu envisagé pour une évolution future.                                   |
| Comptes utilisateurs                                     | <b>Won't Have</b>  | Hors du périmètre du projet en raison de l'absence de backend.                    |
| Classement en ligne                                      | <b>Won't Have</b>  | Nécessiterait une base de données et un système d'authentification.               |



| Fonctionnalité               | Priorité          | Justification                                                          |
|------------------------------|-------------------|------------------------------------------------------------------------|
| Fonctionnalités multijoueurs | <b>Won't Have</b> | Non retenues afin de conserver un périmètre de développement réaliste. |

## Analyse de la priorisation

La priorisation des fonctionnalités a permis de concentrer le développement sur les éléments essentiels de l'application avant d'intégrer des fonctionnalités d'enrichissement. Cette approche a facilité la réalisation d'un **produit minimum viable (MVP)** entièrement fonctionnel dans les délais impartis.

Les fonctionnalités classées **Must Have** constituent le cœur d'AndromeSky et permettent à elles seules de répondre aux objectifs fixés en début de projet. Les fonctionnalités **Should Have** apportent une réelle valeur ajoutée en améliorant l'expérience utilisateur, notamment grâce à l'intégration des APIs externes et à l'enrichissement des informations affichées.

Enfin, plusieurs fonctionnalités ont été identifiées comme des pistes d'amélioration (**Could Have**) mais ont volontairement été reportées afin de privilégier la stabilité de l'application et la finalisation des fonctionnalités principales avant la remise du rapport.

## 4. Justification de la priorisation

La priorisation des fonctionnalités a été réalisée dans le but de garantir la réalisation d'un produit fonctionnel et cohérent. Les fonctionnalités "must have" permettent d'assurer le fonctionnement de base de l'application, tandis que les fonctionnalités "should have" améliorent l'expérience utilisateur et renforcent l'intérêt pédagogique du projet.

Enfin, les fonctionnalités "nice to have" ont été identifiées comme des axes d'amélioration permettant d'enrichir le projet sans compromettre sa réalisation dans les délais.

Cette approche permet de structurer le développement de manière progressive et de limiter les risques liés à une surcharge de fonctionnalités.

## Technologies utilisées et justifications

Le développement de l'application AndromeSky repose sur un ensemble de technologies sélectionnées en fonction des besoins du projet, de leur pertinence technique, de leur facilité d'intégration ainsi que par affinités. Chaque choix a été effectué dans le but de garantir la faisabilité du projet tout en assurant une bonne qualité de développement.

### 1. Angular

Le framework principal utilisé pour le développement de l'application est Angular.

Je l'ai choisi pour plusieurs raisons. Tout d'abord, par affinité personnelle, Angular utilise le paradigme de la programmation orienté objet que je trouve plus élégante en plus d'offrir énormément de possibilité.

Ensuite, il s'agit d'un framework structuré qui permet d'organiser efficacement une application de type SPA (Single Page Application). Il propose une architecture basée sur des composants, facilitant la séparation des responsabilités et la maintenabilité du code.

De plus, Angular intègre nativement de nombreux outils utiles, tels que la gestion de services, le routage ou encore la communication entre composants. Cela permet de structurer le projet de manière claire et évolutive tout en évitant les dépendances entre les différentes parties du projet.

Enfin, l'utilisation d'Angular est un choix logique par rapport à sa popularité dans le monde professionnel et sa robustesse.

---

## 2. TypeScript

L'application est développée en TypeScript, qui est le langage recommandé avec Angular.

TypeScript apporte un typage statique au JavaScript, ce qui permet de détecter plus facilement les erreurs lors du développement. Il améliore également la lisibilité et la maintenabilité du code, en particulier dans un projet comportant plusieurs fonctionnalités et interactions complexes. De plus, il facilite l'auto-complétions de l'IDE.

Ce choix est particulièrement pertinent dans le cadre d'une application structurée comme AndromeSky, notamment grâce au typage des objets astronomiques, des réponses des APIs et des différents modèles de données utilisés par l'application. Cette approche permet de détecter rapidement de nombreuses erreurs pendant le développement et d'améliorer la maintenabilité du code.

---

## 3. Aladin Lite

Pour l'affichage de la carte du ciel, l'application utilise la bibliothèque Aladin Lite.

Ce choix a été motivé par la complexité de la représentation du ciel en temps réel. Développer un moteur graphique complet à partir de zéro, notamment avec HTML Canvas, aurait nécessité un travail important et aurait trop complexifié le projet.

Aladin Lite permet de bénéficier directement d'une carte du ciel interactive, avec des données astronomiques fiables et une gestion des coordonnées intégrée. Cela permet de se concentrer sur la logique métier du projet, notamment les interactions utilisateur et les fonctionnalités pédagogiques.

Ce choix répond également à la recommandation d'utiliser des outils existants afin de se concentrer sur la valeur ajoutée du projet.

Un Proof of Concept (POC) a été réalisé afin de valider l'intégration de la bibliothèque Aladin Lite dans une application Angular.

Ce prototype a permis de vérifier plusieurs éléments techniques essentiels :

- L'affichage correct de la carte du ciel dans Angular
- La gestion native du zoom et du déplacement sur la carte
- La récupération des coordonnées astronomiques (RA/DEC) lors d'un clic utilisateur

- La possibilité de centrer automatiquement la carte sur un objet spécifique

Ces tests ont permis de confirmer qu'Aladin Lite répond aux besoins principaux du projet concernant la visualisation et les interactions utilisateur.

Le POC a également mis en évidence certaines limitations, notamment la dépendance à une connexion Internet pour le chargement des données astronomiques et des surveys externes. En effet, bien que la bibliothèque soit installée localement dans le projet, une partie importante des ressources visuelles est récupérée depuis des serveurs distants spécialisés.

Pour résumer, Aladin Lite est utilisé comme composant central de l'application. Il permet non seulement d'afficher la carte du ciel, mais également de centrer automatiquement la vue sur un objet, de mettre en évidence certaines constellations et de servir de support aux différents modes de quiz proposés.

---

## 4. APIs externes

### [NASA Open APIs](#)

Les **NASA Open APIs** sont utilisées afin d'enrichir la consultation des objets astronomiques.

Lorsqu'une image est disponible pour un objet sélectionné, celle-ci est automatiquement récupérée puis affichée dans le panneau d'informations.

L'utilisation de cette API permet d'illustrer les objets célestes par des photographies réelles provenant de la NASA, renforçant ainsi l'aspect pédagogique de l'application.

### [API Wikipédia](#)

L'API de Wikipédia est utilisée afin de récupérer automatiquement une description synthétique des objets astronomiques.

L'intégration de cette API a nécessité un travail supplémentaire afin de résoudre les nombreuses ambiguïtés présentes dans les résultats de recherche. En effet, plusieurs objets astronomiques partagent leur nom avec des personnes, des entreprises, des lieux ou d'autres sujets sans rapport avec l'astronomie.

Pour améliorer la pertinence des résultats, plusieurs mécanismes ont été mis en place :

- utilisation de plusieurs alias de recherche pour un même objet ;
- analyse des catégories Wikipédia ;
- système de score permettant d'évaluer la pertinence d'une page ;
- filtrage des résultats ambiguës ou incomplets.

Cette approche permet d'obtenir automatiquement une description fiable pour la grande majorité des objets présents dans le catalogue.

## 5. Données locales (JSON)

Une majeure partie des données utilisées dans l'application pour les objets célestes (constellations, objets Messier, etc.), est stockée localement sous forme de fichiers JSON.

Ce choix permet de garder un contrôle total sur les données utilisées, de simplifier la gestion du contenu et d'éviter une dépendance excessive aux services externes.

Les données locales permettent également d'optimiser les performances de l'application, en évitant des requêtes réseau inutiles pour des informations statiques.

---

## 6. Outils de développement

Le développement de l'application est réalisé à l'aide d'un environnement de développement moderne, tel que Visual Studio Code, offrant de nombreuses fonctionnalités facilitant l'écriture du code (auto-complétions, extensions, gestion de projet).

Le versioning du projet est assuré via GitHub, permettant de suivre l'évolution du code, de gérer les différentes versions et d'assurer une sauvegarde du projet.

L'IA sera également utilisée pour faciliter le débogage en cas de blocage persistant.

---

## 7. Conclusion sur les choix technologiques

Les technologies choisies pour le développement de l'application AndromeSky ont été sélectionnées dans une logique de cohérence et d'efficacité. L'utilisation d'Angular permet de structurer l'application, tandis que l'intégration d'Aladin Lite simplifie la visualisation du ciel.

L'ajout d'API externes, comme celles de la NASA, permet d'enrichir le contenu, tandis que l'utilisation de données locales garantit un meilleur contrôle sur les fonctionnalités du projet.

Ces choix permettent de répondre aux objectifs du TFE tout en assurant une bonne qualité de développement et une expérience utilisateur pertinente.

## Outils

### Outils de développement

Le développement du projet AndromeSky nécessite l'utilisation de plusieurs outils permettant de faciliter l'écriture du code, la gestion du projet, le débogage ainsi que le déploiement de l'application. Ces outils ont été choisis afin de proposer un environnement de développement moderne et adapté à la création d'une application web SPA basée sur Angular.

---

#### 1. Visual Studio Code

L'environnement de développement principal utilisé pour le projet est Visual Studio Code.

Cet éditeur de code a été choisi pour plusieurs raisons :

- Compatibilité complète avec Angular et TypeScript
- Nombreuses extensions facilitant le développement web
- Interface légère et personnalisable
- Outils de débogage intégrés
- Terminal intégré permettant d'exécuter les commandes Angular directement depuis l'éditeur

Visual Studio Code permet également d'améliorer la productivité grâce à l'auto-complétions, au formatage automatique du code ainsi qu'à la détection rapide des erreurs de type et de syntaxe.

---

## 2. Angular CLI

Le développement de l'application s'appuie également sur Angular CLI, qui constitue l'outil officiel de gestion des projets Angular.

Angular CLI permet notamment :

- La création rapide de composants et services
- Le lancement d'un serveur de développement local
- La compilation du projet
- La gestion simplifiée de la structure du projet

Cet outil permet de standardiser le développement et de gagner du temps lors de l'implémentation des différentes fonctionnalités.

---

## 3. Git et GitHub

Le versioning du projet est assuré via Git, tandis que l'hébergement du dépôt est réalisé sur GitHub.

L'utilisation de GitHub présente plusieurs avantages :

- Sauvegarde régulière du projet
- Suivi de l'évolution du code
- Gestion des différentes versions
- Possibilité de revenir à une version précédente en cas de problème
- Meilleure organisation du développement

Le projet est structuré autour d'une branche principale ("main") ainsi que de branches secondaires dédiées à certaines fonctionnalités spécifiques.

Cette organisation permet de développer certaines parties de l'application de manière isolée avant de les intégrer dans la version principale du projet.

---

#### 4. Outils de conception et de schématisation

Des outils de création de diagrammes et de maquettes seront utilisés afin de préparer certaines parties du projet et d'illustrer le rapport.

Parmi ceux-ci :

- Draw.io pour les diagrammes UML et les schémas de flux utilisateurs des features
- Figma & ChatGPT pour certaines maquettes d'interface

Ces outils permettent de représenter visuellement le fonctionnement de l'application ainsi que les interactions utilisateur.

---

#### 5. Chrome Developer Tools

Les outils de développement intégrés au navigateur permettent d'analyser le comportement de l'application pendant son exécution. Ils seront utilisés tout au long du développement afin d'inspecter le code HTML et CSS généré, de surveiller les erreurs de script, d'analyser les requêtes réseau vers les différentes APIs et de vérifier les performances de l'application. Ces outils facilitent également le débogage des composants Angular ainsi que la validation du bon fonctionnement des différentes fonctionnalités.

---

#### 6. Postman

Postman est un logiciel permettant de tester et d'analyser les appels aux APIs sans devoir exécuter l'application. Il sera utilisé pour vérifier le fonctionnement des différentes APIs intégrées au projet, notamment celles de la NASA et de Wikipédia, en testant les requêtes HTTP, les paramètres transmis ainsi que les réponses retournées. Cet outil a permis de valider rapidement le format des données reçues et de faciliter le développement des différents services chargés de communiquer avec ces APIs.

---

#### 7. Utilisation de l'intelligence artificielle

L'intelligence artificielle sera également utilisée comme outil d'assistance au développement, notamment pour faciliter le débogage en cas de blocage persistant.

L'utilisation de l'IA ne remplace pas le travail de développement, mais permet :

- D'obtenir des pistes de résolution de problèmes
- De mieux comprendre certaines erreurs techniques
- De gagner du temps lors du débogage
- D'obtenir des explications sur certaines technologies ou méthodes de développement.
- D'éviter de se perdre avec la manipulation de données complexes tels que les données astronomiques d'objets célestes.

Cette approche permet de compléter les ressources de documentation classiques et de faciliter l'apprentissage de certaines notions plus complexes.

---

## Outils d'hébergement

Une fois le développement terminé, l'application devra être hébergée afin d'être accessible depuis un navigateur web. Le choix de la solution d'hébergement s'est porté sur GitHub Pages.

---

### 1. GitHub Pages

L'application AndromeSky sera hébergée via [GitHub Pages](#).

GitHub Pages est un service d'hébergement proposé directement par [GitHub](#) permettant de publier facilement des sites web statiques à partir d'un dépôt Git.

Ce choix présente plusieurs avantages dans le cadre du projet :

- hébergement gratuit
- intégration directe avec le dépôt GitHub du projet
- simplicité de mise en ligne
- possibilité de mettre à jour rapidement l'application
- compatibilité avec les applications Angular de type SPA

Cette solution est particulièrement adaptée aux projets frontend ne nécessitant pas de serveur backend complexe.

---

### 2. Déploiement de l'application Angular

Le déploiement de l'application se fera à partir d'une version de production générée avec Angular.

Le processus de mise en ligne se déroulera en plusieurs étapes :

1. génération du build de production Angular
2. optimisation des fichiers statiques
3. publication automatique sur GitHub Pages
4. mise à disposition de l'application via une URL publique

Cette approche permet de simplifier considérablement le processus de déploiement tout en garantissant une bonne accessibilité de l'application.

---

### 3. Intégration avec GitHub

Le projet étant déjà versionné via Git et hébergé sur GitHub, l'utilisation de GitHub Pages permet de centraliser :

- le code source
- le versioning

- le suivi du développement
- l'hébergement de l'application

Cette centralisation facilite la gestion globale du projet et permet de conserver une organisation claire du développement.

---

#### 4. Compatibilité avec les technologies utilisées

GitHub Pages est compatible avec l'architecture du projet AndromeSky, notamment :

- Angular
- TypeScript
- les fichiers JSON locaux
- les bibliothèques frontend utilisées

L'application reposant principalement sur des technologies frontend et des API externes, aucune infrastructure serveur complexe n'est nécessaire.

Les données dynamiques, telles que les images NASA ou certaines ressources astronomiques, continueront d'être récupérées via des services externes directement depuis le navigateur.

---

#### 5. Limitations de l'hébergement

Bien que GitHub Pages soit adapté au projet, certaines limitations doivent être prises en compte :

- Absence de backend serveur personnalisé
- Dépendance à des API externes
- Impossibilité d'exécuter du code serveur côté hébergement
- Dépendance à une connexion Internet pour certaines ressources astronomiques

Ces limitations restent toutefois compatibles avec les objectifs du projet, l'application étant principalement orientée frontend.

---

#### 6. Gestion du stockage des données

L'application AndromeSky utilise principalement des fichiers **JSON** afin de stocker les données astronomiques nécessaires à son fonctionnement. Ce choix permet de conserver localement les informations relatives aux objets célestes (nom, coordonnées, type, catalogue, constellation, etc.) sans nécessiter de base de données ni de serveur backend.

Cette approche présente plusieurs avantages :

- simplicité d'implémentation ;
- rapidité de chargement des données ;
- facilité de maintenance et d'enrichissement du catalogue ;



- parfaite compatibilité avec un hébergement statique sur GitHub Pages.

Les informations complémentaires, telles que les images et les descriptions, ne sont pas stockées localement mais récupérées à la demande via les APIs de la NASA et de Wikipédia.

Les données générées pendant l'utilisation de l'application (scores, préférences utilisateur, historique, etc.) ne sont pas conservées de manière persistante dans le MVP. Mais pourra faire l'objet d'une amélioration future en fonction du temps de développement.

---

## 7. Conclusion sur l'hébergement

Le choix de GitHub Pages permet de proposer une solution d'hébergement simple, gratuite et parfaitement adaptée à une application Angular de type SPA.

Cette solution facilite le déploiement du projet tout en assurant une bonne intégration avec les outils de développement déjà utilisés, notamment Git et GitHub.

---

## 8. Hébergement des ressources externes

L'application repose également sur plusieurs ressources externes :

- Bibliothèque astronomique Aladin Lite
- API NASA & Wikipédia
- Données locales au format JSON

Certaines ressources sont chargées directement depuis des services externes, tandis que les données statiques nécessaires au fonctionnement du quiz seront intégrées directement dans le projet afin de garantir leur disponibilité.

---

## Conclusion sur les outils utilisés

Les différents outils sélectionnés pour le développement et l'hébergement du projet ont été choisis afin de proposer un environnement de travail moderne, structuré et adapté aux besoins du projet.

L'utilisation conjointe d'Angular, GitHub, des outils de conception et des services d'hébergement permet de garantir une bonne organisation du développement ainsi qu'une mise en ligne simplifiée de l'application.

## Anticipation des problèmes et difficultés potentielles

Le développement de l'application AndromeSky implique l'utilisation de plusieurs technologies et de nombreuses interactions entre différents composants. Bien que le projet ait été conçu de manière progressive et structurée, plusieurs difficultés potentielles ont été identifiées avant le début du développement.

L'anticipation de ces problèmes permet de mieux préparer les différentes étapes du projet et de limiter les risques pouvant ralentir son avancement.

---

## 1. Intégration de la bibliothèque Aladin Lite

L'une des principales difficultés anticipées concerne l'intégration de la bibliothèque Aladin Lite dans une application Angular.

Aladin Lite est une bibliothèque JavaScript externe qui ne repose pas sur l'architecture de composants Angular. Son intégration nécessitera donc une adaptation afin de garantir une bonne communication entre la carte du ciel et les composants de l'application.

Plusieurs éléments devront être pris en compte :

- Récupération des événements utilisateur (clics, déplacements, zoom)
- Récupération des coordonnées astronomiques lors des interactions
- Synchronisation entre la logique Angular et la carte du ciel
- Gestion du cycle de vie des composants Angular

Cette partie du projet représente une difficulté technique importante, car elle implique l'interaction entre plusieurs systèmes différents.

---

## 2. Gestion des coordonnées astronomiques

L'application repose fortement sur les coordonnées astronomiques, notamment l'ascension droite (RA) et la déclinaison (DEC).

Ces coordonnées seront utilisées pour :

- Localiser les objets du quiz
- Vérifier les réponses utilisateur
- Afficher les objets sur la carte du ciel
- Calculer la distance entre deux positions célestes

La manipulation de ces coordonnées représente une difficulté potentielle, notamment en raison de la précision nécessaire pour les calculs et des différences entre les systèmes de coordonnées utilisés en astronomie.

Une mauvaise gestion de ces données pourrait entraîner des erreurs de validation ou des incohérences dans l'affichage des objets.

---

## 3. Gestion des interactions utilisateur

Le mode quiz nécessite plusieurs interactions complexes :

- Détection du clic utilisateur
- Récupération de la position sélectionnée
- Comparaison avec l'objet cible

- Affichage d'un retour visuel adapté

Cette logique doit rester fluide et intuitive pour éviter de rendre l'application difficile à utiliser.

L'équilibre entre précision astronomique et accessibilité utilisateur constitue donc un point d'attention important.

---

## 4. Communication avec les API externes

### [NASA Open APIs & Wikipedia API](#)

L'application utilise les NASA Open APIs afin de récupérer des images et l'API de Wikipédia pour des informations complémentaires.

L'utilisation d'API externes implique plusieurs risques :

- Indisponibilité temporaire du service
- Limitation du nombre de requêtes
- Temps de réponse variables
- Changement éventuel de structure des données retournées
- Réponse incohérente

Afin de limiter l'impact de ces problèmes, certaines données importantes pourront être stockées localement ou chargées de manière différée.

### [Aladin Lite](#)

Le Proof of Concept réalisé avec Aladin Lite a également permis d'identifier certaines contraintes techniques liées à l'utilisation de ressources astronomiques externes.

L'affichage de certaines cartes du ciel repose sur des requêtes chargées depuis des serveurs distants. Cela implique plusieurs limitations potentielles :

- Dépendance à une connexion Internet
- Indisponibilité temporaire de certains serveurs
- Problèmes de politique CORS lors du chargement de certaines ressources
- Temps de chargement variables selon les requêtes utilisés

Ces contraintes devront être prises en compte lors du développement final de l'application, notamment en prévoyant une gestion des erreurs et des mécanismes de *fallback* lorsque certaines ressources externes ne sont pas accessibles.

---

## 5. Gestion de la logique du quiz

Le système de quiz représente une partie centrale du projet. Sa mise en œuvre nécessite :

- La création d'un système de questions cohérent

- La gestion des niveaux de difficulté
- La vérification des réponses
- La gestion des scores et des retours utilisateur

Cette logique devra rester suffisamment flexible pour permettre l'ajout futur de nouveaux modes de jeu, notamment le mode "SpaceGuessR".

La conception de cette architecture constitue donc un enjeu important pour garantir l'évolutivité du projet.

---

## 6. Gestion de l'expérience utilisateur

L'application vise un public relativement large, y compris des utilisateurs ayant peu de connaissances en astronomie.

L'un des défis du projet consiste donc à rendre les interactions suffisamment simples et intuitives tout en conservant une dimension éducative et scientifique.

Plusieurs éléments devront être travaillés dans ce sens :

- Clarté de l'interface
  - Simplicité de navigation
  - Lisibilité des informations affichées
  - Équilibre entre difficulté et accessibilité dans le quiz
- 

## 7. Manque de connaissances sur certaines technologies

Certaines technologies utilisées dans le projet représentent encore des domaines en cours d'apprentissage, notamment :

- Angular
- L'intégration de bibliothèques externes complexes
- La manipulation de données astronomiques
- Certains calculs et manipulations liés aux coordonnées célestes

Ce manque d'expérience peut ralentir certaines étapes du développement, notamment lors de la mise en place de fonctionnalités plus techniques.

Toutefois, ce projet représente également une opportunité d'apprentissage importante permettant de développer de nouvelles compétences techniques.

---

## 8. Gestion du temps et des priorités

Le projet comporte plusieurs fonctionnalités avancées, dont certaines ont volontairement été classées comme optionnelles afin d'éviter une surcharge de travail.

La principale difficulté sera de maintenir une progression stable tout en priorisant les fonctionnalités essentielles.

Une attention particulière devra être portée à :

- La gestion du planning
- Le respect des priorités définies
- L'évitement des fonctionnalités trop complexes en début de développement

Cette approche permettra de garantir la réalisation d'un produit fonctionnel avant d'envisager les fonctionnalités secondaires.

---

## 9. Conclusion sur les difficultés anticipées

Plusieurs difficultés techniques et organisationnelles ont été identifiées avant le début du développement du projet. Ces difficultés concernent principalement l'intégration des technologies externes, la manipulation des données astronomiques ainsi que la conception des interactions utilisateur.

Cependant, l'identification préalable de ces risques permet d'adopter une approche progressive et structurée du développement, en mettant l'accent sur les fonctionnalités essentielles avant d'envisager les extensions plus complexes.

## Architecture de l'application

L'application AndromeSky a été développée selon une architecture modulaire reposant sur les bonnes pratiques recommandées par Angular. Les différentes responsabilités de l'application sont réparties entre des composants, des services, des modèles et des ressources de données, ce qui facilite la maintenance, l'évolution du projet et la réutilisation du code.

L'organisation générale du projet est illustrée par la structure suivante :

```
src/  
└─ app/  
    ├── components/  
    ├── services/  
    ├── models/  
    ├── data/  
    ├── helpers/  
    └─ constants/
```

## Composants

Les composants constituent la partie visible de l'application. Chacun possède une responsabilité bien définie afin de limiter les dépendances entre les différentes parties de l'interface.

Les principaux composants développés sont :

- **Header** : barre de navigation entre les différents modes de l'application.
- **SkyMap** : affichage et interaction avec la carte du ciel grâce à Aladin Lite.
- **SidePanel** : conteneur principal affichant les informations ou les interfaces liées au quiz.
- **ObjectPanel** : présentation détaillée des informations d'un objet astronomique.
- **SearchBar** : recherche rapide d'un objet du catalogue.
- **QuizSettings** : configuration d'une nouvelle partie (mode, difficulté, nombre de questions).
- **QuizQuestions** : affichage des questions et gestion des réponses.
- **QuizResults** : résumé des performances obtenues à la fin d'un quiz.
- **SvgIcon** : affichage des différentes icônes SVG utilisées dans l'interface.

Cette organisation permet de développer chaque fonctionnalité indépendamment tout en conservant une interface claire et facilement maintenable.

---

## Services

La logique métier de l'application est centralisée dans plusieurs services Angular spécialisés.

Chaque service possède un rôle précis :

| Service                          | Responsabilité                                                                     |
|----------------------------------|------------------------------------------------------------------------------------|
| <b>AstronomicalObjectService</b> | Gestion du catalogue des objets astronomiques et génération des questions de quiz. |
| CatalogLoaderService             | Chargement des différents catalogues astronomiques au démarrage de l'application.  |
| JsonLoaderService                | Lecture des fichiers JSON utilisés par les différents catalogues.                  |
| ConstellationService             | Gestion des constellations et des segments permettant leur affichage.              |
| SkyMapService                    | Communication avec Aladin Lite et gestion de l'affichage de la carte du ciel.      |

|                  |                                                                                            |
|------------------|--------------------------------------------------------------------------------------------|
| QuizService      | Gestion complète du déroulement des quiz (questions, progression, validation des réponses) |
| ScoreService     | Calcul et suivi du score du joueur.                                                        |
| WikipediaService | Recherche et sélection des descriptions Wikipédia les plus pertinentes.                    |
| NasaImageService | Récupération des images provenant des APIs de la NASA.                                     |
| AppStateService  | Gestion du mode courant de l'application (exploration ou quiz).                            |

Cette séparation permet de conserver des composants principalement dédiés à l'affichage, tandis que les traitements sont réalisés au sein des services.

---

## Modèles de données

Les données échangées dans l'application sont décrites à l'aide d'interfaces TypeScript regroupées dans le dossier `models`.

Cette approche garantit un typage fort des différentes entités manipulées par l'application, telles que :

- les objets astronomiques ;
- les constellations ;
- les questions de quiz ;
- les scores ;
- les réponses des APIs de la NASA et de Wikipédia.

Le recours aux modèles facilite la maintenance du projet et réduit les risques d'erreurs lors du développement.

---

## Ressources de données

Les catalogues astronomiques sont stockés dans des fichiers JSON situés dans le dossier `public/data`.

Les principaux catalogues utilisés sont :

- Messier
- Hipparcos
- Constellations

Ce choix permet d'ajouter ou de modifier facilement des objets astronomiques sans devoir modifier le code de l'application.

## Helpers et constantes

Afin d'éviter la duplication de code, certaines fonctionnalités communes ont été regroupées dans les dossiers helpers et constants.

Ils contiennent notamment les fonctions utilitaires utilisées lors de la recherche des descriptions Wikipédia ainsi que les constantes nécessaires au fonctionnement de ces traitements.

## Planning du projet

Le développement du projet s'est déroulé sur plusieurs mois. Afin d'organiser le travail et de suivre l'avancement des différentes étapes, un planning prévisionnel a été établi. Celui-ci distingue les phases d'analyse, de développement, de rédaction du rapport et de finalisation du projet.

Diagramme de Gantt – Projet AndromeSky



## Présentation du résultat final

À l'issue du développement, AndromeSky est une application web permettant d'explorer le ciel interactif tout en proposant plusieurs fonctionnalités pédagogiques destinées à faciliter la découverte des objets astronomiques.

Les sections suivantes présentent les principales fonctionnalités développées au cours de ce travail de fin d'études.



## Interface générale

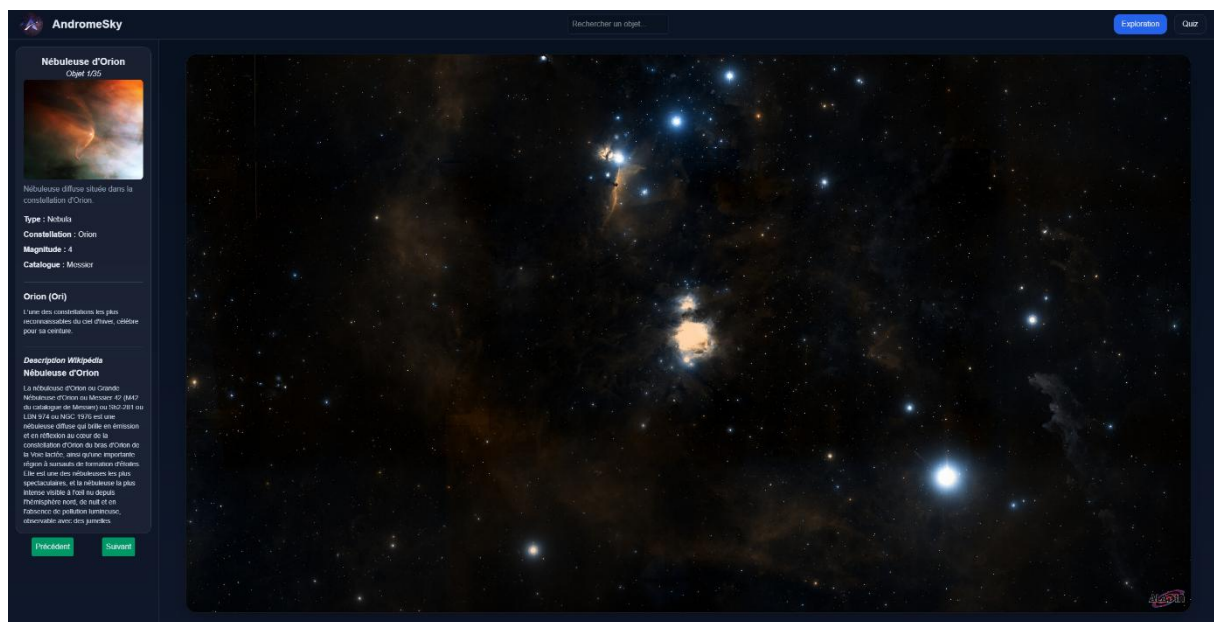


Figure 1 - Interface principale de l'application Andromesky

La Figure 1 présente l'interface principale de l'application. Celle-ci est composée d'une barre de navigation supérieure, d'un panneau latéral affichant les informations relatives à l'objet sélectionné et d'une carte du ciel interactive reposant sur la bibliothèque Aladin Lite.

## Exploration du ciel

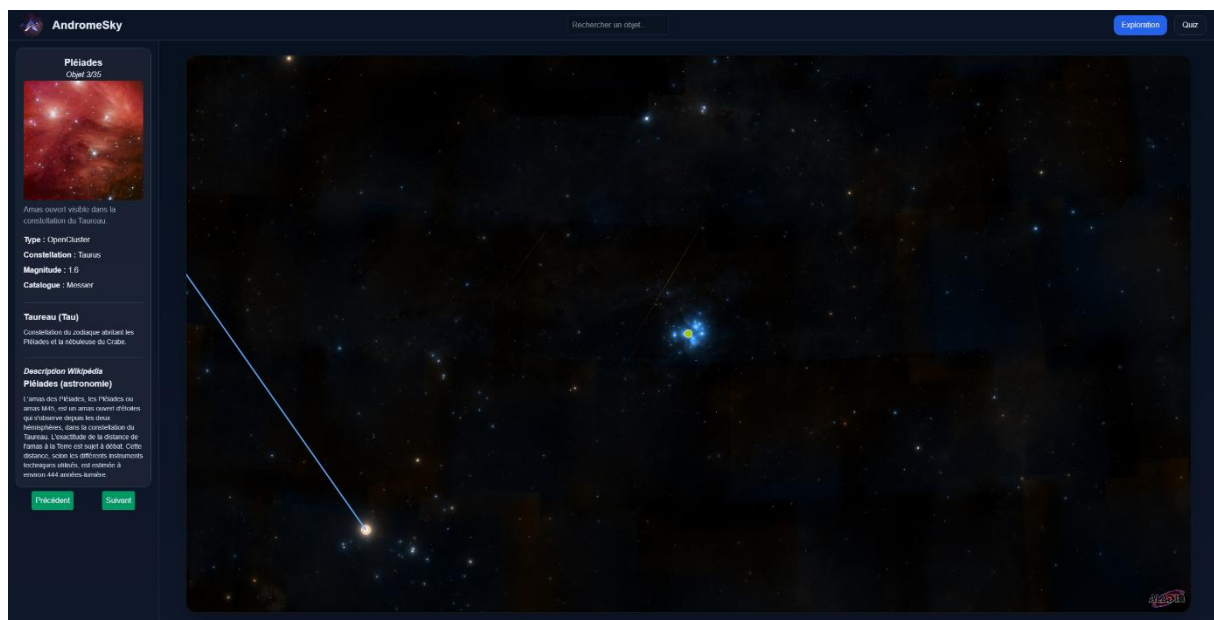


Figure 2 - Consultation d'un objet astronomique

Lorsqu'un objet est sélectionné, le panneau d'information latéral affiche les informations issues des fichiers JSON ainsi qu'une image provenant des APIs de la NASA (si disponible) ainsi que la description Wikipédia (lorsqu'une description est disponible)

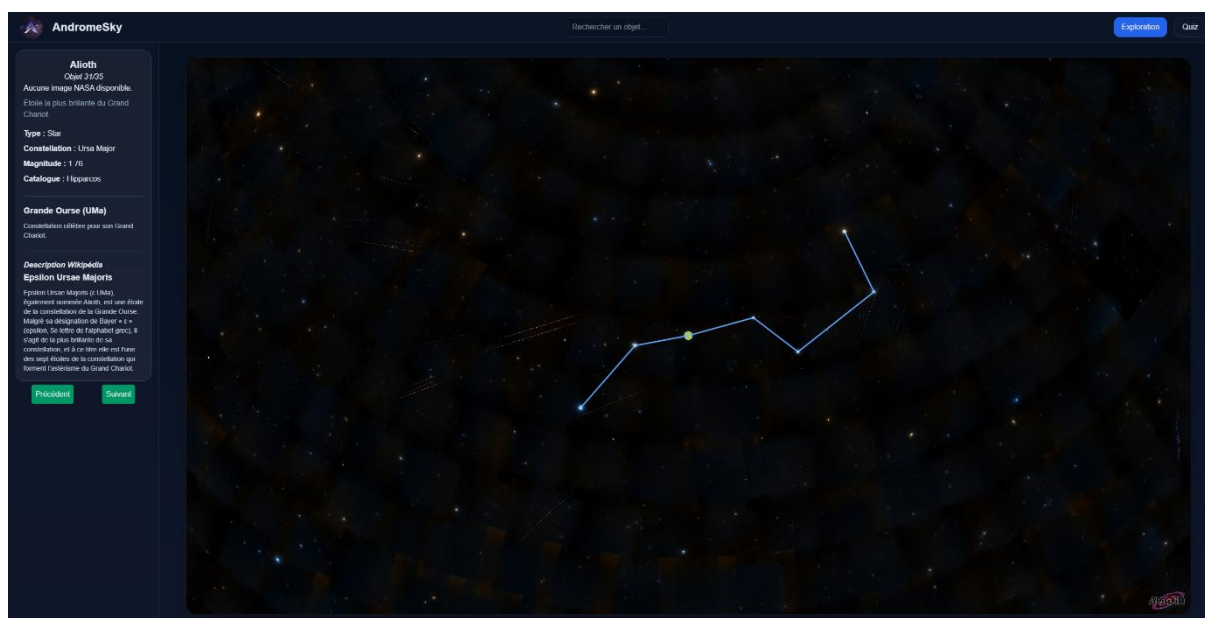


Figure 3 - Mise en évidence d'une constellation

Lorsqu'un objet appartient à une constellation, celle-ci est automatiquement mise en évidence sur la carte afin de faciliter son repérage.

## Recherche

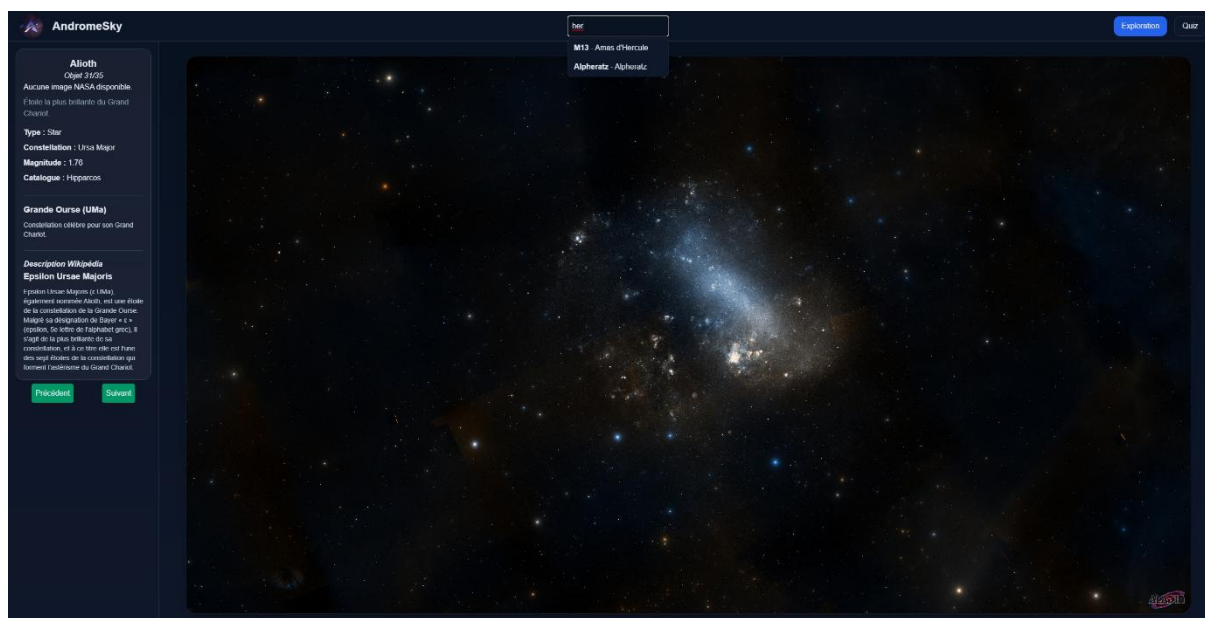


Figure 4 – Recherche d'un objet astronomique

Une barre de recherche avec autocomplétion permet de retrouver rapidement un objet astronomique. Les résultats sont filtrés dynamiquement au fur et à mesure de la saisie.

## Configuration du quiz

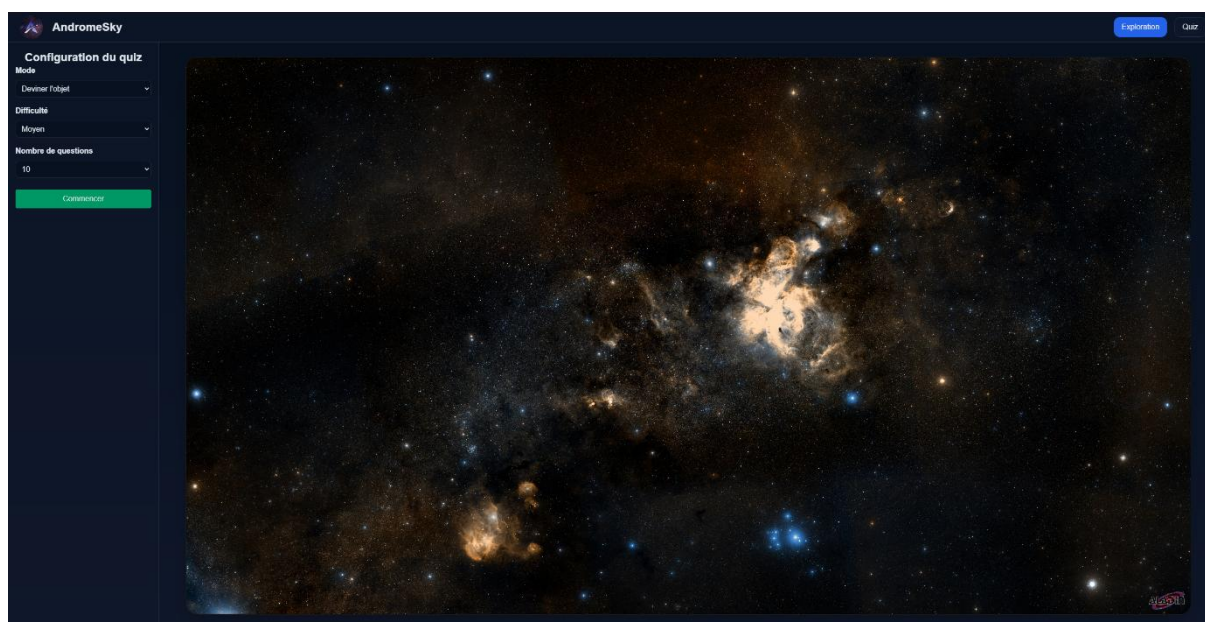


Figure 5 – Configuration d'une partie

L'utilisateur peut choisir son mode de quiz, la difficulté ainsi que le nombre de questions parmi des listes de choix prédéfinis.

## Quiz « Deviner l'objet »

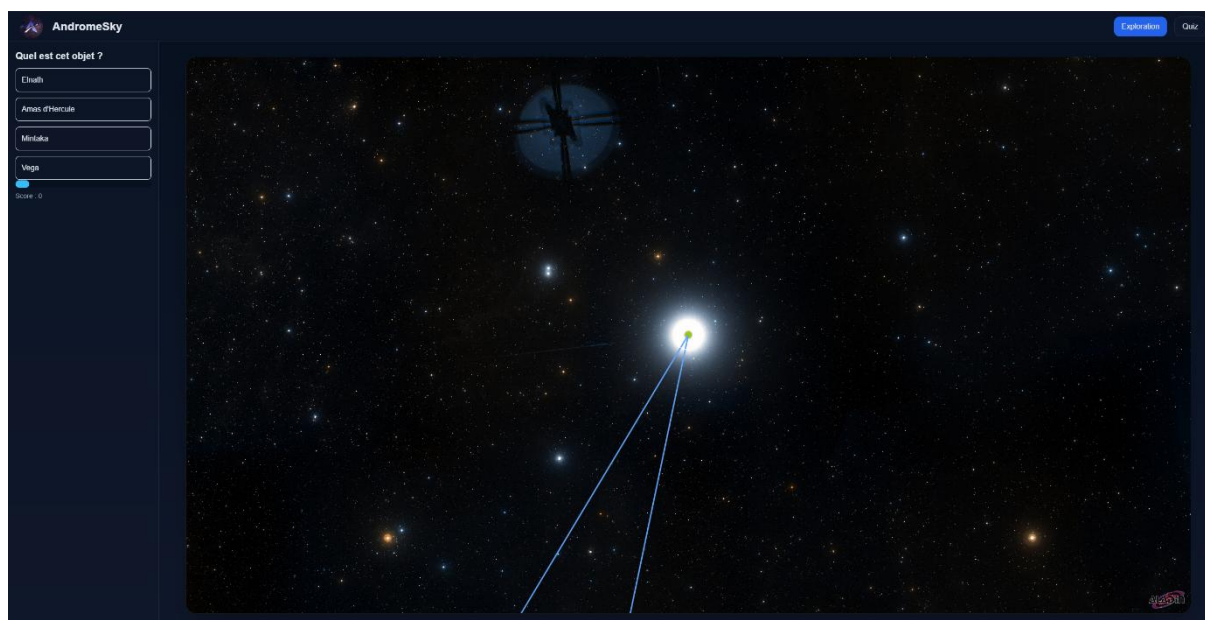


Figure 6 – Mode « Deviner l'objet »

L'utilisateur doit identifier l'objet affiché sur la carte parmi plusieurs propositions.



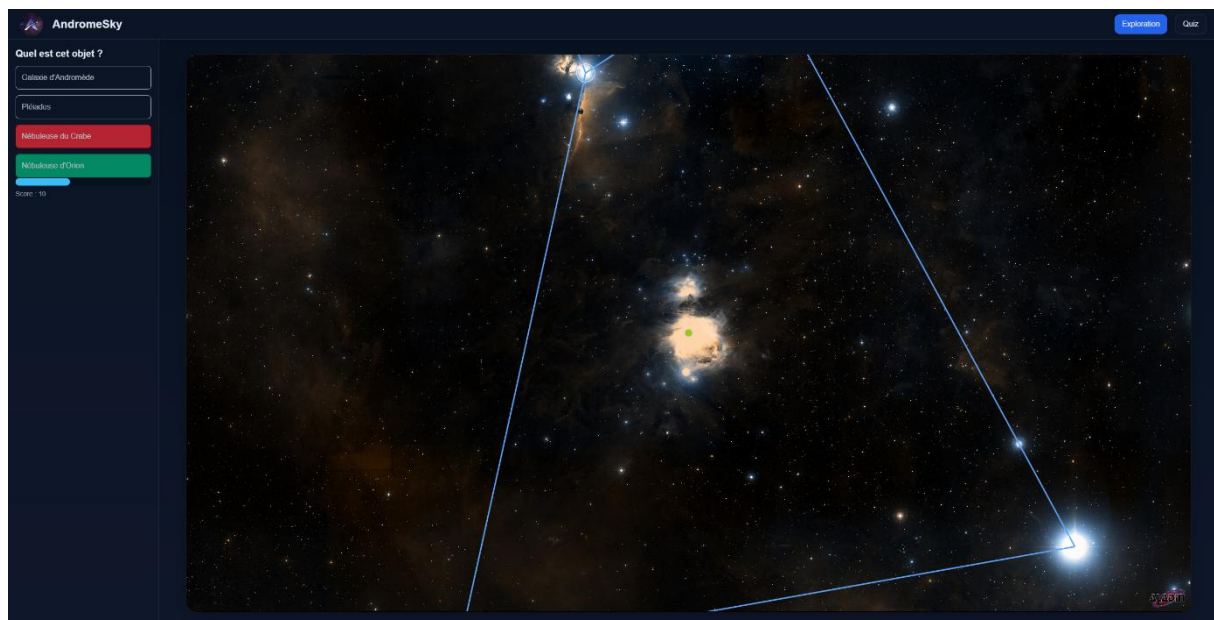


Figure 7 – Retour visuel sur une réponse donnée

Un retour visuel est affiché lors de la validation d'une réponse.

## Quiz « Localiser l'objet »

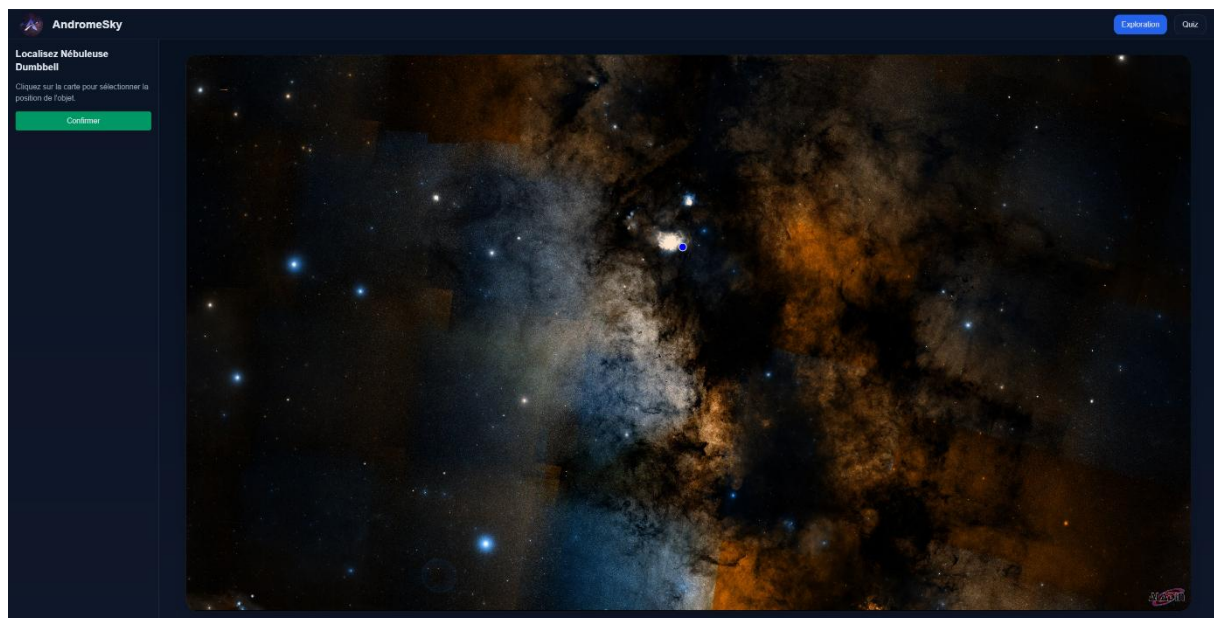


Figure 8 – Mode « Localiser l'objet »

Dans ce second mode, l'utilisateur doit sélectionner directement sur la carte la position de l'objet demandé avant de valider sa réponse.

## Résultats



Figure 9 – Résultats d'une partie

À la fin d'une partie, l'application affiche un récapitulatif comprenant le score obtenu, le nombre de bonnes réponses ainsi que le taux de réussite.

Au terme du développement, l'application **AndromeSky** répond aux principaux objectifs définis lors de l'analyse des besoins. Elle propose une plateforme d'exploration astronomique interactive combinant visualisation, apprentissage et mise en pratique des connaissances.

L'application permet à l'utilisateur de naviguer librement dans une carte du ciel interactive grâce à la bibliothèque **Aladin Lite**. Les objets astronomiques présents dans le catalogue peuvent être recherchés, sélectionnés et explorés à travers une interface simple et intuitive.

Chaque objet dispose d'un panneau d'informations regroupant ses principales caractéristiques ainsi que des informations complémentaires obtenues grâce aux APIs de la NASA et de Wikipédia. Cette approche permet d'offrir un contenu pédagogique plus riche tout en limitant la quantité de données à maintenir localement.

Le projet intègre également deux modes de quiz destinés à évaluer les connaissances de l'utilisateur :

- un quiz à choix multiples consistant à identifier un objet astronomique ;
- un quiz de localisation demandant de retrouver directement un objet sur la carte du ciel.

Ces deux modes peuvent être configurés selon plusieurs niveaux de difficulté et sont accompagnés d'un système de score ainsi que d'un écran récapitulatif présentant les résultats obtenus.

L'ensemble de l'application est développé sous Angular et repose sur une architecture modulaire basée sur des composants, des services spécialisés et des modèles de données fortement typés. Cette organisation facilite la maintenance du projet ainsi que son évolution future.

Le choix d'un hébergement statique sur GitHub Pages répond aux besoins du projet tout en simplifiant son déploiement. Les catalogues astronomiques étant stockés localement au format JSON, aucune infrastructure serveur n'est nécessaire au fonctionnement de l'application.

Dans son état actuel, AndromeSky constitue un **produit minimum viable (MVP)** complet, répondant aux objectifs pédagogiques fixés au début du projet et offrant une base solide pour de futures évolutions.

## Difficultés rencontrées

Le développement d'AndromeSky a donné lieu à plusieurs difficultés techniques qui ont nécessité des recherches et différentes phases d'expérimentation.

L'une des premières difficultés a concerné l'intégration de la bibliothèque **Aladin Lite** au sein d'une application Angular. Au-delà de son intégration dans le cycle de vie des composants Angular, certaines fonctionnalités de la bibliothèque généraient des **erreurs CORS (Cross-Origin Resource Sharing)**. Ces erreurs provenaient directement de certaines requêtes internes effectuées par Aladin Lite vers des services externes. Bien qu'elles n'empêchent pas le fonctionnement de l'application, elles ont nécessité une phase d'analyse afin de vérifier qu'elles n'avaient pas d'impact sur les fonctionnalités développées et qu'elles étaient indépendantes du code de l'application.

Une autre difficulté est apparue au cours du développement avec l'apparition occasionnelle d'une erreur **"Out of memory"** lors de l'exécution de l'application. Cette erreur semblait se produire de manière aléatoire et n'a jamais pu être reproduite de façon fiable malgré de nombreux essais. Plusieurs pistes ont été explorées, notamment la structure des données astronomiques, le chargement des catalogues et l'utilisation de la mémoire par le navigateur. Faute de scénario reproductible, il n'a toutefois pas été possible d'identifier précisément l'origine du problème. L'erreur a continué occasionnellement à se produire au fil du développement, sans qu'une cause unique puisse être clairement établie.

La récupération automatique des descriptions Wikipédia s'est également révélée plus complexe que prévu. De nombreux objets astronomiques possèdent des homonymes appartenant à d'autres domaines (personnes, entreprises, lieux, œuvres culturelles, etc.). Il a donc été nécessaire de développer un système de sélection reposant sur plusieurs critères, notamment l'utilisation d'alias de recherche, l'analyse des catégories Wikipédia et un système de score de pertinence permettant de sélectionner automatiquement la page la plus adaptée. Ce système bien qu'efficace n'est pas infaillible, surtout sur un catalogue d'objet céleste qui finira par regrouper des centaines d'objets.

De même, la recherche d'image via l'API de la Nasa peut renvoyer des images qui ne sont pas représentatives de l'objet sélectionné.

Enfin, l'évolution progressive du projet a conduit à plusieurs refactorisations de l'architecture afin de conserver une séparation claire entre les composants d'interface, les services métier et les modèles de données. Cette réorganisation a facilité l'intégration des nouvelles fonctionnalités tout en maintenant une architecture cohérente et facilement maintenable.

## Perspectives d'évolution

Bien que l'application AndromeSky réponde aux objectifs fixés pour ce travail de fin d'études, plusieurs pistes d'amélioration pourraient être envisagées afin d'enrichir l'expérience utilisateur et d'étendre les fonctionnalités proposées.

L'une des premières évolutions consisterait à enrichir le catalogue astronomique en intégrant de nouveaux catalogues tels que **NGC**, **Caldwell** ou encore les **planètes** du système solaire. L'ajout de nouveaux objets permettrait de diversifier les possibilités d'exploration et d'augmenter progressivement la difficulté des différents modes de quiz.

Le système de score pourrait également être amélioré par l'ajout d'une **persistance locale** des meilleurs résultats, permettant aux utilisateurs de suivre leur progression au fil du temps. Des statistiques plus détaillées, telles que le taux de réussite par catégorie d'objet ou par niveau de difficulté, pourraient également être proposées.

Le mode **Quiz de localisation** pourrait être enrichi en tenant compte de la proximité entre la position sélectionnée par l'utilisateur et la position réelle de l'objet. Un système de score plus précis permettrait ainsi de récompenser les réponses proches de la bonne localisation, même lorsqu'elles ne sont pas parfaitement exactes.

Enfin, plusieurs améliorations ergonomiques pourraient être apportées à l'interface, notamment une meilleure distinction entre les interactions de clic et de déplacement sur la carte du ciel, ainsi que des animations et retours visuels renforçant l'expérience utilisateur.

Grâce à son architecture modulaire, AndromeSky constitue une base solide permettant d'intégrer facilement ces évolutions dans de futures versions du projet.

## Conclusion

Le développement d'AndromeSky m'a permis de mettre en pratique les connaissances acquises au cours de ma formation tout en découvrant de nouvelles technologies et en approfondissant plusieurs aspects du développement d'applications web modernes.

L'objectif initial était de concevoir une application permettant de découvrir le ciel de manière interactive tout en proposant une approche pédagogique de l'astronomie. Au fil du développement, le projet a progressivement évolué afin d'offrir une expérience plus complète, intégrant non seulement l'exploration libre d'une carte du ciel, mais également des informations enrichies grâce aux APIs de la NASA et de Wikipédia, ainsi que plusieurs modes de quiz destinés à favoriser l'apprentissage.

La réalisation de ce projet m'a également permis de développer mes compétences techniques. J'ai notamment approfondi ma maîtrise d'Angular, de TypeScript et de l'organisation d'une application reposant sur une architecture modulaire composée de composants, de services et de modèles de données. L'intégration de bibliothèques externes, la consommation d'APIs REST, la gestion de fichiers JSON et la résolution de problèmes techniques rencontrés tout au long du développement ont constitué une expérience particulièrement enrichissante.

Au-delà des aspects techniques, ce travail de fin d'études m'a également sensibilisé à l'importance de l'organisation d'un projet informatique. La définition des priorités, la planification du développement,

la rédaction progressive de la documentation ainsi que les différentes phases de tests et de refactorisation ont joué un rôle essentiel dans l'aboutissement du projet.

Le résultat obtenu répond aux principaux objectifs fixés au début du travail de fin d'études et constitue un produit minimum viable (MVP) fonctionnel. L'architecture mise en place permettra d'intégrer facilement de futures améliorations, telles que l'ajout de nouveaux catalogues astronomiques, l'enrichissement des fonctionnalités pédagogiques ou encore l'amélioration du système de quiz.

De plus, ce projet représente une expérience particulièrement formatrice. Il m'a permis de consolider mes compétences en développement frontend, d'acquérir davantage d'autonomie dans la résolution de problèmes complexes et de mener un projet complet, depuis l'analyse des besoins jusqu'à sa réalisation, sa documentation et la préparation de sa présentation.

Enfin, au-delà des compétences techniques que ce travail m'a permis de renforcer, il m'a également fait prendre conscience de l'importance de l'organisation et du respect d'un planning. Avec le recul, je considère ne pas avoir toujours été suffisamment rigoureux sur ces aspects et j'aurais souhaité pouvoir aller encore plus loin dans le développement de l'application. Cette expérience m'a surtout appris qu'il est particulièrement difficile d'estimer avec précision la charge de travail d'un projet informatique et de mesurer, dès le départ, ce qu'il est réellement possible de réaliser dans un temps imparti.

